

BOSS直聘前端一面面经

BOSS直聘一面面经 时长:60min

1. 简单介绍下你的项目
2. 低代码平台（组件）中是如何处理报错的(面试官应该是想问的埋点监控问题)-
3. 拖拽组件是如何实现的（本身的github仓库上有这个组件）
4. 平常是怎么学习的之类的
5. 如何判断当前dom是否被包含（contain和 递归判断parentDom）
6. 追问 如何比较两个dom是否相等
7. 为什么要写这些开源项目和组件库
8. 写的自定义指令中 为什么 要先插入个input输入框 然后选中？
9. 实现中为什么要用废弃api document.execCommand 不使用最新的
10. 懒加载图片指令中 entry.isIntersecting 干嘛的 追问unobserve 卸载监听 img加载资源完成和失败的方法
11. scrollToDom函数干啥的（滚动到指定的dom位置）,如何实现一个过渡动画
12. 平常会看书吗？coding时间占比如何
13. 为什么使用nest.js 做你的后台服务，（这部分跟面试官说了不是很熟），追问 dto是干啥的
14. 平常会写博客吗？发个链接看看，不定高的虚拟列表实现原理，列表滚动过程中是否会抖动？

反问环节

1. 面试官对我的后续发展情况有什么建议
2. 岗位工作内容、方向之类的

面试官nice!和面试官比较合的来，已约二面。

面试问题回答：

1. 简单介绍下你的项目：

- 我的项目是一个低代码平台，旨在通过拖拽组件和可视化配置来快速构建Web应用。平台包括丰富的预定义组件库、灵活的插件系统、实时预览和代码生成功能，帮助开发者和非技术人员以较少的编码快速构建复杂的应用程序。

2. 低代码平台（组件）中是如何处理报错的：

在低代码平台中，处理报错通常需要考虑以下几个方面，以确保系统的稳定性和用户体验：

a. 全局错误捕获：

- 使用全局错误捕获机制来处理未被捕获的错误。例如，可以使用 `window.onerror`、`window.onunhandledrejection` 捕获全局的同步和异步错误。这种方式确保即使在应用中的某个具体功能模块出错，也能及时捕获并记录下来，避免整个应用崩溃。

b. 组件级别的错误边界：

- 在React或Vue等框架中，可以通过实现错误边界（Error Boundary）组件来捕获和处理子组件的渲染错误。错误边界会在组件渲染时捕获到错误，并可以显示备用的UI或者提示用户错误信息，同时不会影响其他部分的正常运行。

c. 埋点监控和日志记录：

- 在应用中集成埋点监控工具（如Sentry、LogRocket等），对用户的操作和系统的错误进行跟踪和记录。埋点可以在用户操作过程中收集上下文信息，帮助开发者分析问题发生的原因和频率。记录的日志可以包括错误堆栈、用户操作路径、设备信息等，方便后续排查和修复问题。

d. 友好的错误提示：

- 当错误发生时，向用户展示友好的错误提示而非技术性错误信息。通过定制化的错误页面或通知，向用户解释问题可能的原因，并提供下一步的建议或解决办法，如重新加载页面、联系支持等。

e. 自动恢复机制：

- 对于一些常见的错误，可以设计自动恢复机制。例如，自动重试请求、回退到安全状态或者重新加载组件。这些措施可以在一定程度上提高系统的健壮性，减少因为偶发性错误导致的用户体验不佳。

f. 开发环境中的详细调试信息：

- 在开发环境中，报错信息通常更详细，以帮助开发者快速定位问题。包括完整的堆栈跟踪、变量状态、网络请求详情等。在生产环境中，这些详细信息可以隐藏或简化，防止敏感信息泄露。

g. 报警和通知：

- 对于关键性的错误，可以配置报警机制，如通过邮件、短信或IM工具通知开发团队，确保问题在第一时间得到响应和处理。

通过以上措施，低代码平台中的报错处理可以做到及时捕获、合理处理、及时通知，确保平台的稳定性和用户体验。

3. 拖拽组件是如何实现的：

- 拖拽组件通常通过HTML5的 `drag and drop` API或者使用第三方库（如 `react-dnd`）来实现。核心步骤包括：定义拖拽的元素及其拖拽事件、设置拖拽的目标位置、处理拖拽开始、移动和结束事件，并更新DOM或数据状态。

4. 平常是怎么学习的之类的：

- 我的学习方法包括：通过阅读技术博客和文档来了解新技术，参加在线课程进行系统学习，做开源项目实践新知识，参与社区讨论获取不同观点。同时，我还会定期复盘项目中的问题，深入理解原理。

5. 如何判断当前DOM是否被包含：

- 可以使用 `Node.contains()` 方法来判断一个DOM元素是否包含另一个DOM元素。此外，还可以通过递归判断父元素是否匹配来实现类似的功能。

6. 如何比较两个DOM是否相等：

- 判断两个DOM是否相等可以通过直接比较DOM对象引用来实现，如 `dom1 === dom2`。如果要比较内容，可以遍历DOM树，逐层比较各个属性和子元素。

7. 为什么要写这些开源项目和组件库：

- 写开源项目和组件库的动机通常包括：分享知识、积累经验、提升编程技能、获取社区反馈，以及建立个人品牌。开源也有助于解决实际问题，方便他人复用解决方案。

8. 写的自定义指令中为什么要先插入个input输入框然后选中：

- 在自定义指令中插入 `input` 输入框并选中，可能是为了实现某种用户输入的聚焦效果，或者确保用户在特定条件下能够直接输入内容，提供更好的用户体验。

9. 实现中为什么要用废弃API `document.execCommand` 不使用最新的：

- `document.execCommand` 虽然被标记为废弃，但在某些浏览器中仍然是最简单的实现特定功能（如剪贴板操作）的方法。使用它可能是为了兼容性或者简单性，但需要注意未来可能需要更新实现。

10. 懒加载图片指令中 `entry.isIntersecting` 干嘛的，追问 `unobserve` 卸载监听 `img` 加载资源完成和失败的方法：

10.1 `entry.isIntersecting` 的作用

`entry.isIntersecting` 是 `IntersectionObserverEntry` 对象的一个属性，它表示目标元素是否与根元素的交叉区域相交。简单来说，当目标元素进入或离开视口（或指定的根元素）时，`entry.isIntersecting` 的值会相应地变为 `true` 或 `false`。

在懒加载图片的指令中，`entry.isIntersecting` 通常用于判断图片是否进入了视口。如果 `entry.isIntersecting` 为 `true`，意味着图片已经进入视口，此时可以开始加载该图片资源；如果为 `false`，则图片在视口之外，不需要加载。

```

1 javascript
2 复制代码
3 const observer = new IntersectionObserver((entries, observer) => {
4     entries.forEach(entry => {if (entry.isIntersecting) {// 图片进入视口, 开始
        加载图片const img = entry.target;
5         img.src = img.dataset.src;
6         observer.unobserve(img); // 加载后停止观察
7     }
8 });
9 });
10
11 // 为每个懒加载图片元素添加观察
12 images.forEach(img => {
13     observer.observe(img);
14 });

```

10.2 unobserve 卸载监听的作用

`IntersectionObserver.unobserve()` 方法用于停止观察指定的目标元素。当图片加载完成后, 已经不再需要继续监控该图片的可见性, 因此可以使用 `unobserve` 来停止观察, 释放资源, 优化性能。

在懒加载图片的场景中, 当图片进入视口并成功开始加载后, 通常会立即调用 `observer.unobserve(img)`, 这样当用户滚动页面时, 浏览器不会再为已经加载的图片触发多余的观察事件。

```

1 javascript
2 复制代码
3 observer.unobserve(img); // 停止观察该图片

```

10.3 img 加载资源完成和失败的方法

在图片的懒加载过程中, 我们需要处理图片资源加载成功和失败的情况。可以通过以下两个事件来处理:

- **onload**: 当图片资源成功加载时触发。可以在这个事件中执行一些操作, 比如标记图片加载完成, 或为图片添加一些视觉效果。
- **onerror**: 当图片加载失败时触发。可以在这个事件中处理加载失败的情况, 比如显示一个默认的占位图或者重试加载。

示例代码如下:

```

1 javascript

```

```
2 复制代码
3 img.onload = () => {console.log('Image loaded successfully');// 在此处处理加载成功的逻辑, 例如添加样式、移除loading动画等
4 };
5 img.onerror = () => {console.error('Failed to load image');// 在此处处理加载失败的逻辑, 例如设置默认占位图
6     img.src = 'path/to/placeholder.jpg';
7 };
```

总结

在懒加载图片的实现中, `entry.isIntersecting` 用于判断图片是否进入视口, 从而决定是否加载图片; `unobserve` 用于在图片加载后停止观察, 优化性能; `onload` 和 `onerror` 则分别处理图片资源加载成功和失败的情况, 确保用户得到最佳体验。

11. `scrollToDom` 函数干啥的, 如何实现一个过渡动画:

- `scrollToDom` 函数用于滚动页面到指定的DOM位置。实现过渡动画可以通过 `requestAnimationFrame` 或者CSS中的 `scroll-behavior: smooth;` 属性来实现平滑滚动效果。

12. 平常会看书吗? coding时间占比如何:

- 我通常会通过阅读技术书籍来深入学习某些主题, 并通过实际编码来实践这些知识。我的时间分配可能是: 70%的时间在编码, 30%的时间在学习和研究新技术。

13. 为什么使用Nest.js做你的后台服务, 追问DTO是干啥的:

- 我选择Nest.js作为后台服务框架, 因为它是一个基于Node.js的进程框架, 具有良好的模块化、依赖注入和类型安全特性。DTO (Data Transfer Object) 用于在控制器和服务层之间传递数据, 确保数据的结构化和类型安全。

14. 平常会写博客吗? 发个链接看看, 不定高的虚拟列表实现原理, 列表滚动过程中是否会抖动?

- 我会通过写博客来记录学习笔记和开发心得。不定高的虚拟列表通常通过设置可滚动容器的高度, 并动态渲染列表中的可见元素, 避免一次性渲染所有元素。抖动问题可能来自于列表项的动态加载或渲染, 需要通过优化渲染逻辑或使用防抖策略来缓解。